

Taller 4

Se ha encomendado a los estudiantes de la asignatura de fundamentos de computación de alto desempeño realizar una mejora en el cálculo del algoritmo de PI, utilizando programación paralela

Pseudocódigo

```
Proceso ValorDePi
  pi <- 0;
  Escribir Sin Saltar "Ingresa el valor de n:";
  Leer n;
  Para i<-1 Hasta n Con Paso 1 Hacer
    Escribir "PROCESO ", i;
    pi <- pi+4.0/(i*2-1);
    Escribir "";
  FinPara
  Escribir "Valor de pi: ", pi;
FinProceso
```

Para ello debe escribir la rutina en cpp para la solución de PI y registrar los siguientes casos:

- 1) Calcular el valor de PI para una precisión con el valor $n = 1000$, determinando el tiempo de procesamiento normal y luego compararlo con el procesamiento paralelo
- 2) Calcular el valor de PI para una precisión con el valor $n = 10000$, determinando el tiempo de procesamiento normal y luego compararlo con el procesamiento paralelo
- 3) Calcular el valor de PI para una precisión con el valor $n = 3$, ¿es mejor el procesamiento paralelo?

Formato de Entrega: Word con Pantallazos de código, explicación de código y resultados, en conjunto con las conclusiones.

Explicación del algoritmo

El algoritmo de la serie de potencias de PI se refiere a una serie matemática convergente cuyo límite tiende a PI. Existen varias series de potencias utilizadas para calcular PI, pero una de las más conocidas y simples es la serie de Leibniz, que es una

serie alternante que converge a $\pi/4$. La serie de Leibniz se define de la siguiente manera:

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 + \dots$$

Esta serie converge lentamente a π , lo que significa que se necesitan muchos términos para obtener una aproximación precisa de π . Sin embargo, existen otras series que convergen más rápidamente, como la serie de Bailey-Borwein-Plouffe (BBP) y la fórmula de Chudnovsky. Estas series permiten calcular dígitos específicos de π sin calcular los dígitos previos y se utilizan en cálculos de alta precisión de π .

A continuación, se muestra un ejemplo de código en C++ para calcular π utilizando la serie de Leibniz:

```
#include <stdio.h>
#include <time.h>
int main() {
    clock_t inicio, fin;
    double time_ms;
    inicio = clock();
    int suma=0;
    for(int i=0; i<4000; i++){
        suma = suma + i;
        printf("%d\n", i);
    }
    printf("La suma es: %d\n", suma);
    fin = clock();
    time_ms = ((double)(fin-inicio)/CLOCKS_PER_SEC)*1000;
    printf("Tiempo : %.2f milisegundos", time_ms);
    return 0;
}
```

Este código calcula una aproximación de π utilizando la serie de Leibniz con un número fijo de términos. La convergencia de la serie de Leibniz es lenta, y se necesitan un gran número de términos para obtener una aproximación precisa de π . Para algoritmos más rápidos y precisos, considera investigar la serie BBP o la fórmula de Chudnovsky.

Ahora tienes que incorporar openMP al programa y demostrar que ante más procesadores y más paralelización funciona mejor o peor.

Recuerda:

```
g++ -fopenmp main.cpp -o main
```

Y luego, ejecuta el programa:

```
./main
```